A Self-Supervised Transformer for Direction Prediction in Polymarket BTC Binary Options

Gareth Flynn
University College London
COMP0162 Advanced Machine Learning in Finance, 2025/26

Abstract

We apply a self-supervised Transformer encoder to direction prediction in Polymarket binary Up/Down contracts on Bitcoin (BTC) that settle every five minutes. The encoder is pre-trained on unlabelled market sequences via masked-feature reconstruction, then fine-tuned with causal attention to produce per-timestep probabilities that the contract settles "Up". We evaluate two trading rules: a probability-threshold baseline and an expected-P&L edge rule that enters only when the model disagrees with the market price by a meaningful margin. On out-of-sample contracts (15% temporal split) the Transformer attains statistically indistinguishable accuracy from a logistic-regression baseline at every horizon, yet achieves a significant improvement in downstream trading P&L and Sharpe ratio. The Transformer's advantage is not directional accuracy, which is tied with the baseline at every horizon, but a better-calibrated probability surface, yielding more profitable and more selective entries.

1 Introduction

Prediction markets are exchange-traded venues where participants buy and sell contingent claims that aggregate probabilistic beliefs about the outcome of future events, with prices functioning as crowdsourced probability estimates that have been shown to outperform traditional forecasting [Wolfers and Zitzewitz, 2004]. Polymarket, a decentralised prediction market built on the Polygon blockchain, gained widespread attention during the 2024 US Presidential election and now sees daily trading volumes exceeding $400M across thousands of active markets. Following demand for short-term cryptocurrency price contracts, Polymarket introduced recurring five-minute BTC price-direction contracts in February 2026. Each contract resolves to $1 (up) or $0 (down) at settlement, generating a short sequential decision problem: when to enter a long or short position within the five-minute window, and at what price. Classic supervised learning can estimate the direction probability at a single snapshot but cannot optimise the entry policy over a sequence.

The model observes 60 timesteps of eight features and must produce a calibrated probability at every step. Logistic regression collapses the time series to a single snapshot; recurrent models capture order but suffer from sequential bottlenecks; attention-based architectures resolve both limitations [Vaswani et al., 2017]. We adopt a two-stage Transformer pipeline inspired by self-supervised representation learning in language [Devlin et al., 2019] and time series [Zerveas et al., 2021, Lim et al., 2021]: the encoder is pre-trained via masked-feature reconstruction, then fine-tuned with causal attention. Related work has applied BERT-style pre-training to limit-order books [Sirignano and Cont, 2019] and Temporal Fusion Transformers to multi-horizon forecasting [Zhou et al., 2021, Lim et al., 2021]; but no published study has, to our knowledge, applied self-supervised Transformers to blockchain-native binary prediction markets.

Our contributions are: (i) we collect and feature-engineer a dataset of 15,182 Polymarket BTC five-minute contracts from February–April 2026, with 49.7M contract trades and 911k five-second BTC bars reconstructed from Binance aggregate trades; (ii) we adapt BERT's masked-feature pre-

training [Devlin et al., 2019] to prediction-market microstructure with a small (≈72k parameter) Transformer; and (iii) we show that even where raw classification accuracy is statistically tied with a logistic-regression baseline, the Transformer's temporal calibration yields a significant improvement in trading P&L and Sharpe ratio.

2 Methodology

2.1 Problem Formulation

Each contract is a sequence of $T = 60$ observations at five-second intervals. At step t the model observes a feature vector $\mathbf{x}_t \in \mathbb{R}^8$ and outputs $\hat{p}_t = P(\text{Up} \mid \mathbf{x}_{0:t})$ using only past information; the contract resolves to $y \in \{0, 1\}$.

Trading uses an expected-P&L rule rather than a raw probability threshold. At each step t in the entry window the agent computes

$$E[\text{long}] = (\hat{p}_t - \text{ask}_\uparrow) \cdot \$100, \tag{1}$$

$$E[\text{short}] = ((1 - \hat{p}_t) - \text{ask}_\downarrow) \cdot \$100, \tag{2}$$

against the \$100 notional; it enters on the first t where $\max(E[\text{long}], E[\text{short}]) > \tau_\$$ (we use $\tau_\$ = \$20$ throughout the evaluation). The rule is direction-symmetric and applied identically to both the Transformer and the logistic baseline, so differences in trading performance are attributable purely to differences in the underlying probability estimates $\hat{p}_t$ rather than to any difference in execution logic. Position size is fixed at one unit (\$100 per contract) so that the P&L comparison in Section 3 isolates the machine-learning contribution.

Realised per-episode P&L for a long entry is $(\mathbf{1}\{y = 1\} - \text{ask}_\uparrow) \cdot \$100$ and for a short entry is $(\mathbf{1}\{y = 0\} - \text{ask}_\downarrow) \cdot \$100$. The reference price at each bin is the last executed Polymarket trade price. To simulate realistic execution costs, we add a half-spread of \$0.005 (or \$0.01 at whole-cent ticks) to obtain the ask price a taker would pay. This is conservative: every entry in the backtest is assumed to pay the spread, regardless of whether the underlying trade was a passive (maker) fill that would not have incurred it, so the reported P&L is a lower bound.

2.2 Transformer Encoder

We denote the encoder as a function that takes a length-T sequence of d_{in}-dimensional feature vectors and returns a length-T sequence of d_{model}-dimensional contextualised representations. It consists of an input projection $\mathbf{W}_{\text{in}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{in}}}$, additive sinusoidal positional encoding [Vaswani et al., 2017], and N_L stacked Transformer encoder layers each composed of multi-head self-attention and a position-wise feed-forward network with GELU (Gaussian Error Linear Unit) [Scardapane, 2024] activation. Scaled dot-product attention is defined as:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} \odot \mathbf{M}\right)\mathbf{V}, \tag{3}$$

where $\mathbf{M}$ is an attention mask applied element-wise. During pre-training $\mathbf{M}$ is the identity mask (all ones), so every position attends to every other position (bidirectional attention). During fine-tuning $\mathbf{M}$ is a causal mask with $M_{ij} = 0$ for $j > i$ and $M_{ij} = 1$ otherwise, so that future positions are zeroed out inside the softmax before normalisation. The fine-tuned model therefore only attends to past information at every timestep—a property essential for any deployment that respects causality. Table 1 lists the architecture hyperparameters; the model totals ≈72k trainable parameters.

2.3 Stage 1—Self-Supervised Pre-training

Following BERT [Devlin et al., 2019], we randomly mask a fraction $\rho = 0.15$ of timesteps in each training sequence by replacing the input vector with a learned mask embedding $\mathbf{e}_M \in \mathbb{R}^{d_{\text{in}}}$. The encoder must reconstruct the original feature vectors at the masked positions from the unmasked context. Let $\mathcal{M} \subset \{0, \ldots, T-1\}$ be the masked indices and $\mathbf{h}_t = f_\theta(\tilde{\mathbf{x}})_t$ the encoder output, where $\tilde{\mathbf{x}}$ is the masked input. A linear head g_ϕ maps $\mathbf{h}_t$ back to feature space. The pre-training loss is the mean squared error over masked positions:

$$\mathcal{L}_{\text{pre}}(\theta, \phi) = \frac{1}{|\mathcal{M}|} \sum_{t \in \mathcal{M}} \|g_\phi(\mathbf{h}_t) - \mathbf{x}_t\|_2^2. \tag{4}$$

Table 1: Model architecture and training hyperparameters.

Hyperparameter	Value
Input dimension d_{in}	8
Model dimension d_{model}	64
Attention heads	4
Encoder layers N_L	2
Feed-forward dimension	128
Dropout	0.1
Sequence length T	60
Mask ratio ρ (pre-train)	0.15
Batch size	128
Total parameters	$\approx$72k

This task forces the encoder to learn temporal correlations in the contract price, the BTC price and the order-flow features without ever seeing a label, producing general-purpose representations that transfer to the downstream classification head and reduce the amount of labelled data required for fine-tuning. The mask embedding ensures the model can distinguish missing positions from genuine zero values. We minimise this loss using AdamW [Scardapane, 2024], an Adam variant that decouples weight decay from the gradient update to improve generalisation (lr $= 1 \times 10^{-3}$, weight decay $= 1 \times 10^{-4}$), with a cosine-annealing schedule for up to 30 epochs with early stopping (patience 5) on validation MSE.

2.4 Stage 2—Direction Fine-tuning

We attach a small classification head to the pre-trained encoder: a linear layer maps $\mathbf{h}_t \in \mathbb{R}^{d_{\text{model}}}$ to a scalar logit, followed by a sigmoid producing $\hat{p}_t = \sigma(\mathbf{w}^\top \mathbf{h}_t + b)$. Predictions are made at every timestep using causal attention so that $\hat{p}_t$ only depends on $\mathbf{x}_{0:t}$. We train with a time-weighted binary cross-entropy:

$$\mathcal{L}_{\text{ft}} = \frac{1}{T} \sum_t w_t \big[-y \log \hat{p}_t - (1-y) \log(1-\hat{p}_t) \big], \tag{5}$$

where $w_t \in [0.5, 1.5]$ grows linearly with t. Early predictions (when little information is available) are penalised less, encouraging the model to express genuine uncertainty rather than memorise label-leakage patterns. To mitigate representation collapse during fine-tuning, we freeze the encoder for the first five epochs (linear probing of the head) and then unfreeze it with a discriminative learning-rate schedule (encoder lr $= 3 \times 10^{-5}$, head lr $= 3 \times 10^{-4}$). Training runs up to 40 epochs with early stopping (patience 7) on validation BCE.

2.5 Baseline—Logistic Regression

As a null baseline we train an L_2-regularised logistic regression on the same eight features evaluated at three decision-relevant timesteps ($t = 12, 24, 36$, corresponding to 1, 2 and 3 minutes into the contract). At each timestep it maps the raw feature vector through a linear combination $\hat{p} = \sigma(\boldsymbol{\beta}^\top \mathbf{x}_t)$ with an L_2 penalty to prevent overfitting. Unlike the Transformer, the logistic regression sees only a single timestep's features and cannot attend to the sequence's evolution over time, making it a strong sanity check on the value of temporal modelling.

2.6 Data and Pre-processing

We collect 15,185 BTC five-minute Up/Down contracts from Polymarket's Gamma and CLOB APIs spanning 12 February to 5 April 2026, alongside 49.7M contract trade events. We additionally aggregate every Binance BTCUSDT aggregate trade into 911,280 five-second OHLCV bars, providing $12\times$ finer granularity than the one-minute candles available from both exchanges' historical endpoints. Three contracts with insufficient activity are dropped, leaving 15,182 episodes.

For each contract we compute eight features at each five-second bin: (1, 2) the last executed Up/Down token trade price; (3) intra-contract momentum $\Delta(p_{\text{up}} - p_{\text{down}})$; (4) BTC five-second log return

$\log(\text{BTC}_t/\text{BTC}_{t-1})$; (5) cumulative BTC deviation $\log(\text{BTC}_t/\text{BTC}_0)$ from contract start; (6) net-flow imbalance $(V_{\text{up}} - V_{\text{down}})/(V_{\text{up}} + V_{\text{down}})$; (7) trade intensity normalised by a twelve-bin rolling mean; and (8) a whale signal from Polymarket trades exceeding \$200 notional.

Features are z-score normalised using statistics from only the first twelve bins (one minute) of each training episode, which precedes significant directional price movement and avoids contamination from the extreme near-\$0/\$1 values that appear once the outcome is largely decided. Momentum and BTC deviation use full-episode statistics as they are zero at $t = 0$. The dataset is sorted chronologically and split 70 / 15 / 15 into 10,627 / 2,277 / 2,278 contracts with no future leakage. The label distribution is near-balanced (50.4% Up) and the median bin contains ≈60 trades.

3 Results

3.1 Direction Prediction

Table 2 reports test-set directional accuracy at three decision-relevant timesteps ($t = 12, 24, 36$). The eight features carry modest but rising signal ($65.8\% \to 71.3\% \to 76.7\%$ for the Transformer, $67.3\% \to 72.0\% \to 77.1\%$ for the logistic), and the two models are statistically indistinguishable at every horizon. McNemar tests at $t = 12, 24, 36$ yield p-values of 0.07, 0.27 and 0.45 respectively. Bootstrap 95% confidence intervals overlap at all three horizons: $[63.9\%, 67.8\%]$ vs $[65.2\%, 69.1\%]$ at t=12, $[69.4\%, 73.1\%]$ vs $[70.1\%, 73.8\%]$ at t=24, and $[75.0\%, 78.4\%]$ vs $[75.4\%, 78.8\%]$ at t=36. Direction accuracy alone is therefore not where the Transformer earns its keep; the advantage instead shows up in the confidence trajectory, which we exploit via the edge rule.

Table 2: Test-set directional accuracy at three decision-relevant horizons. The two models are statistically indistinguishable (McNemar $p > 0.07$ at all horizons; bootstrap 95% CIs overlap).

Model	**Acc** t=12 **(1 min)**	**Acc** t=24 **(2 min)**	**Acc** t=36 **(3 min)**
Logistic regression	67.3%	72.0%	77.1%
Transformer (ours)	65.8%	71.3%	76.7%

3.2 Trading Performance

The picture changes dramatically once we move from raw accuracy to trading P&L. We evaluate both models under two trading rules. The first is a naive probability-threshold rule (enter long at the first t with $\hat{p} > 0.80$, short symmetrically). The second is the $E[\text{PnL}]$-based edge rule of Section 2.1 ($\tau_\$ = \$20$, fixed unit size), which only enters when the model's expected profit against the market ask exceeds \$20. Because the same rule is applied identically to both models, any difference in trading P&L isolates the machine-learning contribution.

Table 3 reports results under both rules. Under the threshold rule, both models trade on nearly every contract (2,175 and 2,199 entries) with similar win rates (83.8% vs 81.3%), but the Transformer earns 85% more P&L (\$19,071 vs \$10,319) because its probabilities lead to entries at more favourable prices. Under the edge rule, the Transformer enters 976 contracts and earns \$27,184 at a 72.0% win rate and Sharpe 225.8, while the logistic enters only 294 contracts and earns \$5,919 at a 70.7% win rate and Sharpe 148.9. The two models have essentially identical per-trade margins (\$27.85 vs \$20.13), but the Transformer finds $3.3\times$ more tradable situations, resulting in $4.6\times$ more total P&L. The edge rule lifts the Transformer's total P&L by 42% and its Sharpe by $3.0\times$, and lifts the logistic's Sharpe by $3.8\times$ off a much smaller P&L base.

An independent two-sample t-test on per-trade P&L (976 Transformer trades vs 294 logistic trades) yields $t = 2.83$ with $p = 0.005$, decisively rejecting the null of equal mean per-trade P&L. A per-episode paired t-test (assigning zero P&L to contracts where a model does not enter) confirms the result ($t = 13.34$, $p = 3.7 \times 10^{-39}$). The Transformer's entries are direction-symmetric (564 long, 412 short) as are the logistic's (139 long, 155 short), so the gap is not driven by a directional bias in either model.

Table 4 segments the Transformer's edge-rule entries by entry window. The mean entry step is 28.4 (2.4 min), later than the logistic's 24.4 (2.0 min) because the Transformer's temporally-aware probability takes longer to build conviction but produces a more differentiated signal. Per-trade

Table 3: Out-of-sample trading performance on 2,278 test contracts (15% temporal split). Both models are evaluated under two trading rules: a naive probability threshold ($p > 0.80$) and the $E[\text{PnL}] > \$20$ edge rule, both at a fixed \$100 unit size. Sharpe ratio is annualised assuming $365 \times 288 = 105{,}120$ five-minute contracts per year (crypto 24/7).

Strategy	Trades	Win rate	Total P&L	Avg/trade	Sharpe
Logistic + threshold ($p > 0.80$)	2,199	81.3%	+\$10,319	+\$4.69	39.0
Logistic + $E[\text{PnL}]$ rule	294	70.7%	+\$5,919	+\$20.13	148.9
Transformer + threshold ($p > 0.80$)	2,175	83.8%	+\$19,071	+\$8.77	74.8
Transformer + $E[\text{PnL}]$ rule (ours)	976	72.0%	+\$27,184	+\$27.85	225.8

economics are nearly identical once an entry fires, so the total-P&L gap comes almost entirely from entry density—the Transformer crosses the edge threshold on $3.3\times$ more contracts. Errors concentrate in late-contract entries where BTC reverses in the final 30–60 s and in thin contracts whose flow features are dominated by one or two large trades.

Table 4: Transformer test-set entries under the $E[\text{PnL}]$ edge rule, segmented by entry minute.

Entry window	Trades	Win rate	Avg P&L	Avg entry price
1–2 min	445	73.9%	+\$26.24	0.477
2–3 min	268	66.4%	+\$25.62	0.408
3–4 min	134	66.4%	+\$25.16	0.413
≥ 4 min	129	82.9%	+\$40.87	0.421

4 Discussion

The central finding is that the Transformer's edge does not appear in raw classification accuracy—a logistic regression matches it at every horizon—but in the shape of its probability trajectory. Under the same edge rule, the Transformer finds $3.3\times$ more tradable entries than the logistic with nearly identical per-trade economics, so the P&L gap comes entirely from entry density.

Mechanistically, self-attention lets the model relate any two timesteps directly, recognising patterns such as escalating trade volume accompanying a price move—patterns invisible to a single-snapshot model. The masked pre-training objective acts as a denoising autoencoder over multivariate time series, teaching the encoder what normal market trajectories look like so that the downstream head can detect predictive deviations. The causal mask at fine-tuning ensures deployability: confidence at each step depends only on past observations.

From a finance standpoint, the model's edge is partly circular: a substantial portion of the signal comes from the contract price itself, which already reflects consensus belief. The Transformer adds value by combining this with order-flow features—net-flow imbalance, trade intensity and the whale signal—which empirically carry the strongest marginal signal (the order-flow block dominates by several standard deviations in an ablation, while the cumulative BTC deviation varies by only 0.8σ per contract). The model is not generating fundamental BTC forecasts but rather forming a better-calibrated $P(\text{Up})$ from the combined signal, and the edge rule exploits situations where the market price has not fully absorbed it.

Limitations. The bidirectional attention used during pre-training means encoder weights were optimised with full-context information, though no labels were observed; a stricter alternative would be autoregressive pre-training. Other limitations include the simplified spread model (constant half-spread), the relatively short evaluation window ($\approx$52 days, one BTC regime), and light validation-set search rather than systematic hyperparameter tuning.

Future work. Contrastive self-supervised objectives [Chen et al., 2020] may produce stronger representations than masked reconstruction for noisy financial time series. A learned execution policy (entry, exit and re-entry) trained via policy gradient on top of the frozen direction model could capture

gains from early exits. Transfer experiments to other Polymarket markets (sports, elections) would test whether the learned microstructure prior generalises beyond BTC.

References

Wolfers, J. and Zitzewitz, E. (2004). Prediction markets. *Journal of Economic Perspectives*, 18(2):107–126.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30:5998–6008.

Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *NAACL*.

Zerveas, G., Jayaraman, S., Patel, D., Bhamidipaty, A., and Eickhoff, C. (2021). A transformer-based framework for multivariate time series representation learning. In *KDD*.

Lim, B., Arık, S. Ö., Loeff, N., and Pfister, T. (2021). Temporal Fusion Transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting*, 37(4):1748–1764.

Sirignano, J. and Cont, R. (2019). Universal features of price formation in financial markets: perspectives from deep learning. *Quantitative Finance*, 19(9):1449–1459.

Zhou, H., Zhang, S., Peng, J., Zhang, S., Li, J., Xiong, H., and Zhang, W. (2021). Informer: Beyond efficient transformer for long sequence time-series forecasting. In *AAAI Conference on Artificial Intelligence*.

Chen, T., Kornblith, S., Norouzi, M., and Hinton, G. (2020). A simple framework for contrastive learning of visual representations. In *ICML*.

Scardapane, S. (2024). *Alice's Adventures in a Differentiable Wonderland*. MIT Press.